Chapter 2. How Python Runs Programs

Chương 2. Cách Python thực thi các chương trình

This chapter and the next cover program execution—how you launch code and how Python runs it. In this chapter, we'll begin by studying how the Python interpreter executes programs in general, from an abstract vantage point. With that perspective in hand, Chapter 3 will dive into the nuts and bolts of getting your own programs up and running.

Chương này và chương tiếp theo sẽ trình bày về quy trình thực thi chương trình – cách thức khởi chạy mã nguồn và cách mà Python thực hiện việc thực thi đó. Trong chương này, chúng ta sẽ bắt đầu bằng việc tìm hiểu cách mà trình thông dịch Python thực thi các chương trình, từ góc độ trừu tượng. Sau đó, chúng ta sẽ tìm hiểu chi tiết về các bước cần thực hiện để có thể chạy được các chương trình của riêng mình.

Startup details are inherently platform specific, and some of the material in these two chapters may not apply to the ways you'll be using Python, so you should feel free to skip parts not relevant to your goals. Likewise, readers who have used similar tools in the past and prefer to get to the meat of the language quickly may want to file some of these chapters away for future reference. For the rest of us, let's take a brief look at the way that Python will run our code, before we get into the details of writing or running it.

Thông tin chi tiết về cách vận hành các chương trình được viết bằng Python thường phụ thuộc vào nền tảng cụ thể mà chương trình đó được chạy trên đó. Một số nội dung trong hai chương này có thể không phù hợp với cách bạn sử dụng ngôn ngữ Python. Vì vậy, bạn hoàn toàn có thể bỏ qua những phần không liên quan đến mục tiêu của mình. Tương tự, những độc giả đã từng sử dụng các công cụ tương tự trước đây và muốn nhanh chóng tìm hiểu cách sử dụng ngôn ngữ Python có thể lưu lại những chương này để tham khảo sau này. Còn đối với chúng ta, hãy cùng tìm hiểu sơ lược cách mà Python thực hiện việc chạy các chương trình, trước khi đi vào chi tiết về cách viết và chạy chúng.

Introducing the Python Interpreter

Giới thiệu về trình thông dịch Python

So far, we've mostly been talking about Python as a programming language. In its most widely used form, though, it's also a software system called an *interpreter*. An interpreter is a kind of program that executes other programs. When you write a Python program, the Python interpreter reads your program and carries out the instructions it contains. In effect, the interpreter is a layer of logic between your code and the computer hardware on your machine.

Cho đến nay, chúng ta chủ yếu thảo luận về Python như một ngôn ngữ lập trình. Tuy nhiên, trong hình thức được sử dụng phổ biến nhất, Python còn là một hệ thống phần mềm được gọi là trình thông dịch. Trình thông dịch là loại chương trình dùng để thực thi các chương trình khác. Khi bạn viết một chương trình bằng Python, trình thông dịch Python sẽ đọc chương trình đó và thực hiện các lệnh được ghi trong đó. Về cơ bản, trình thông dịch đóng vai trò như một "lớp trung gian" giữa mã nguồn của bạn và phần cứng máy tính.

When Python is installed, it generates a number of components—minimally, an interpreter and a support library. Depending on how you use it, the Python interpreter may take the form of an executable program, or a set of libraries linked into another program. Depending on which flavor of Python you run, the interpreter itself may be implemented as a C program, a set of Java classes, or something else. Whatever form it takes, the Python code you write must always be run by this interpreter. And to enable that, you usually must install a Python interpreter on your computer.

Khi Python được cài đặt, nó tạo ra nhiều thành phần khác nhau – ít nhất là một trình thông dịch và một bộ thư viện hỗ trợ. Tùy vào cách sử dụng, trình thông dịch Python có thể tồn tại dưới dạng một chương trình có thể chạy được, hoặc dưới dạng một bộ thư viện được tích hợp vào các chương trình khác. Tùy vào phiên bản Python được sử dụng, trình thông dịch có thể được triển khai dưới dạng chương trình viết bằng ngôn ngữ C, các lớp trong ngôn ngữ Java, hoặc dưới dạng khác. Dù ở dạng nào đi nữa, mã nguồn Python mà chúng ta viết đều phải được thực thi bởi trình thông dịch này. Để có thể sử dụng Python, bạn cần phải cài đặt trình thông dịch Python trên máy tính của mình.

You don't need to install Python at this point unless you want to work along with the sole trivial example coming up, and this book won't assume that you've got a Python ready to go until the next chapter. When you're ready, Python installation details vary by platform and are discussed briefly in Chapter 3, and covered in full in Appendix A.

Bạn không cần phải cài đặt Python vào lúc này, trừ khi bạn muốn thực hành với ví dụ đơn giản được đưa ra trong sách. Cuốn sách này sẽ không giả định rằng bạn đã cài đặt Python sẵn sàng để sử dụng cho đến chương tiếp theo. Khi bạn sẵn sàng, hướng dẫn cài đặt Python sẽ khác nhau tùy theo nền tảng. Thông tin chi tiết về việc cài đặt Python sẽ được trình bày ngắn gọn trong Chương 3, và chi tiết hơn trong Phụ lục A.

Program Execution Thực thi chương trình

What it means to run a Python script depends on whether you look at this task as a programmer, or as a Python interpreter. Both views offer important perspectives on Python programming.

Ý nghĩa của việc thực thi một script Python phụ thuộc vào việc chúng ta nhìn nhận nhiệm vụ này từ góc độ của một lập trình viên, hay từ góc độ của trình thông dịch Python. Cả hai quan điểm này đều mang lại những góc nhìn quan trọng trong việc lập trình bằng Python.

The Programmer's View Góc nhìn của lập trình viên

In its simplest but most common form, a Python program is just a text file containing Python statements. For example, the file listed in Example 2-1, named *script0.py*, may be one of the most trivial Python scripts one could dream up, but it passes for a fully functional Python program.

Ở dạng đơn giản nhất nhưng cũng phổ biến nhất, một chương trình Python chỉ là một tệp văn bản chứa các câu lệnh Python. Ví dụ, tệp script0.py được đề cập trong Ví dụ 2-1 chính là một ví dụ điển hình về chương trình Python đơn giản. Tuy nhiên, đây vẫn là một chương trình Python hoàn chỉnh có thể được sử dụng thực tế.

Example 2-1. script0.py Ví dụ 2-1. script0.py

```
print('hello world')
print(2 ** 100)
```

This file contains two Python `print` statements, which simply print a string (the text in quotes) and a numeric expression result (2 to the power 100) to the output stream (the GUI or window where the file is run, normally). Don't worry about the syntax of this code yet—for now, we're interested only in how it runs. This book will explain the `print` statement, and why you can raise 2 to the power 100 so easily in Python, in its later chapters.

Tệp tin này chứa hai câu lệnh `print` trong ngôn ngữ Python. Các câu lệnh này chỉ đơn giản là in ra một chuỗi ký tự (nội dung được đặt trong dấu ngoặc kép) và kết quả của một phép tính số học (2 lũy thừa 100) vào luồng đầu ra (giao diện người dùng nơi tệp tin được thực thi). Bạn không cần lo lắng về cú pháp của đoạn mã này vào lúc này; điều chúng ta quan tâm là cách mà đoạn mã này hoạt động. Trong các chương tiếp theo của cuốn sách này, chúng tôi sẽ giải thích chi tiết về câu lệnh `print`, cũng như lý do tại sao việc tính 2 lũy thừa 100 lại dễ dàng đến vậy trong Python.

You can create such a file of statements with any text editor you like; see Appendix A for suggestions. By convention, Python program files are given names that end in *.py*; technically, this naming scheme is required only for files that are "imported"—a term clarified in the next chapter—but most Python files have *.py* names for consistency.

Bạn có thể tạo ra các tệp chứa các câu lệnh như vậy bằng bất kỳ trình soạn thảo văn bản nào mà bạn thích; xem Phụ lục A để biết thêm gợi ý. Theo quy ước, các tệp chương trình Python thường có đuôi là .py. Về mặt kỹ thuật, quy ước này chỉ áp dụng cho các tệp được "import" vào chương trình – khái niệm này sẽ được giải thích rõ hơn trong chương tiếp theo. Tuy nhiên, vì tính nhất quán, hầu hết các tệp Python đều có đuôi .py.

After you've typed these statements into a text file, you must tell Python to *execute* the file—which simply means to run all the statements in the file from top to bottom, one after another. As you'll see in the next chapter, you can launch Python program files by typing command lines, by clicking their icons, from within coding GUIs, and with other techniques. If all goes well, when you execute the file, you'll see the results of the two `print` statements show up somewhere on your computer—usually and by default, in the same window you were in when you ran the program:

Sau khi nhập các câu lệnh này vào tệp văn bản, bạn cần yêu cầu Python thực thi tệp đó. Điều này có nghĩa là Python sẽ thực hiện từng câu lệnh trong tệp, theo thứ tự từ trên xuống dưới. Như bạn sẽ thấy trong chương tiếp theo, bạn có thể khởi chạy các chương trình Python bằng cách nhập các lệnh vào dòng lệnh, nhấp vào biểu tượng của chúng, hoặc sử dụng các giao diện người dùng đồ họa. Nếu mọi thứ diễn ra suôn sẻ, khi bạn thực thi tệp, kết quả của các câu lệnh `print` sẽ được hiển thị trên máy tính của bạn – thường là trong cùng cửa sổ mà bạn đã sử dụng để chạy chương trình.

```
hello world
1267650600228229401496703205376
```

For example, here's what happened when this script was run in a Command Prompt window with a command line on a Windows laptop, to make sure it didn't have any silly typos:

Ví dụ, đây là những gì xảy ra khi đoạn mã này được chạy trong cửa sổ Command Prompt trên một chiếc máy tính xách tay chạy hệ điều hành Windows, nhằm đảm bảo rằng không có lỗi chính tả nào xảy ra:

```
C:\Users\me\code> py script0.py
hello world
1267650600228229401496703205376
```

See Chapter 3 for the full story on this process, especially if you're new to programming; we'll get into all the details of writing and launching programs there. For our purposes here, we've just run a Python script that prints a string and a number. We probably won't attract venture capital or go viral on GitHub with this code, but it's enough to capture the basics of program execution.

Xem Chương 3 để biết chi tiết về quy trình này. Đặc biệt, nếu bạn là người mới bắt đầu học lập trình, chương này sẽ cung cấp đầy đủ thông tin về cách viết và chạy các chương trình. Trong ví dụ này, chúng ta chỉ đơn giản chạy một đoạn mã Python để in ra một chuỗi ký tự và một con số. Có lẽ đoạn mã này sẽ không thu hút được vốn đầu tư hay trở nên nổi tiếng trên GitHub, nhưng nó vẫn đủ để giúp chúng ta hiểu được những nguyên lý cơ bản của việc thực thi chương trình.

Python's View Quan điểm của Python

The brief description in the prior section is fairly standard for scripting languages, and it's usually all that most new Python programmers need to know. You type code into text files, and you run those files through the interpreter. Under the hood, though, a bit more happens when you tell Python to "go." Although knowledge of Python internals is not strictly required for Python programming, having a basic understanding of Python's runtime structure up front can help you grasp how your code fits into the bigger picture of program execution.

Mô tả ngắn gọn ở phần trên khá là điển hình đối với các ngôn ngữ lập trình. Đó cũng chính là những gì mà hầu hết các lập trình viên Python mới bắt đầu cần biết. Bạn chỉ cần gõ mã nguồn vào các tệp văn bản, sau đó chạy các tệp đó thông qua trình thông dịch. Tuy nhiên, ẩn sau đó còn nhiều quy trình phức tạp hơn nữa khi bạn ra lệnh cho Python thực hiện công việc. Mặc dù việc hiểu rõ cấu trúc nội bộ của Python không phải là điều bắt buộc để lập trình bằng Python, nhưng việc nắm được cơ bản về cấu trúc thời gian thực của Python sẽ giúp bạn hiểu rõ hơn cách mã nguồn của mình hoạt động trong quá trình thực thi chương trình.

When you instruct Python to run your script, there are a few steps that Python carries out before your code actually starts crunching away. Specifically, it's first compiled to something called "bytecode" and then routed to something called a "virtual machine." This holds true only for the most common version of Python, and you'll meet variations on this model in a moment. Since the most common is, well, most common, let's see how this works first.

Khi bạn yêu cầu Python chạy đoạn mã của mình, có vài bước mà Python thực hiện trước khi đoạn mã đó bắt đầu được thực thi. Cụ thể, đoạn mã sẽ được biên dịch thành dạng gọi là "bytecode", sau đó được chuyển đến một "máy ảo" để được thực thi. Quy trình này chỉ áp dụng cho phiên bản Python phổ biến nhất. Bạn sẽ gặp nhiều biến thể của quy trình này trong các phiên bản Python khác. Vì phiên bản phổ biến nhất vẫn là phiên bản được sử dụng phổ biến nhất, vậy thì hãy cùng tìm hiểu cách thức hoạt động của quy trình này trước.

Bytecode compilation Quá trình biên dịch mã bytecode

Internally, and almost completely hidden from you, when you execute a program Python first compiles your *source code* (the statements in your text file) into a format known as *bytecode*. Compilation is simply a translation step, and bytecode is a lower-level, platform-independent representation of your source code. Roughly, Python translates each of your source statements into a group of bytecode instructions by decomposing them into individual steps. This bytecode translation is performed to speed execution—bytecode can be run much more quickly than the original source code statements in your text file.

Về mặt nội bộ, và gần như hoàn toàn không được hiển thị cho người dùng, khi bạn thực thi một chương trình bằng Python, trước tiên Python sẽ biên dịch mã nguồn của bạn (những câu lệnh trong tệp văn bản) thành một định dạng gọi là bytecode. Quá trình biên dịch thực chất chỉ là bước chuyển đổi mã nguồn sang dạng khác. Bytecode là dạng biểu diễn mã nguồn ở cấp độ thấp hơn, và không phụ thuộc vào nền tảng cài đặt. Nói một cách đơn giản, Python chuyển đổi từng câu lệnh trong mã nguồn thành các lệnh bytecode bằng cách phân tích chúng thành các bước nhỏ hơn. Việc chuyển đổi này nhằm mục đích tăng tốc độ thực thi chương trình – bytecode có thể được thực thi nhanh hơn nhiều so với các câu lệnh nguồn trong tệp văn bản.

You'll notice that the prior paragraph said that this is *almost* completely hidden from you. If the Python program has write access on your machine, it will save the bytecode of your programs in files that end with a *.pyc* extension (*.pyc* means *.py* source, compiled). Technically, this save doesn't happen when running a single file as we did in the preceding section but does for all but the topmost file in meatier multifile programs (more on this in a moment).

Bạn sẽ nhận thấy rằng đoạn văn trước đã nói rằng thông tin này gần như hoàn toàn bị ẩn khỏi tầm nhìn của người dùng. Nếu chương trình Python có quyền ghi vào máy tính của bạn, nó sẽ lưu các đoạn mã nguồn của chương trình dưới dạng các tệp có phần mở rộng .pyc. Về mặt kỹ thuật, việc lưu trữ này không xảy ra khi chỉ chạy một tệp duy nhất, như chúng ta đã làm ở phần trước. Tuy nhiên, việc lưu trữ này lại xảy ra với tất cả các tệp trong chương trình, ngoại trừ tệp đầu tiên trong chương trình đó. (Chúng ta sẽ tìm hiểu thêm về điều này sau.)

Python saves its bytecode files in a subdirectory named *__pycache__* located in the directory where your source files reside, and in files whose names identify the Python version that created them (e.g., *script0.cpython-312.pyc* for 3.12). The *__pycache__* subdirectory avoids clutter, and the naming convention for bytecode files prevents different Python versions installed on the same computer from overwriting each other's saved bytecode. We'll study this bytecode file model in more detail in Chapter 22, though it's automatic and irrelevant to most Python programs and is free to vary among the alternative Python implementations described ahead.

Python lưu các tệp bytecode của mình trong một thư mục con có tên __pycache__. Thư mục này nằm cùng thư mục chứa các tệp nguồn của bạn. Tên các tệp bytecode sẽ cho biết phiên bản Python nào đã tạo ra chúng (ví dụ: script0.cpython-312.pyc tương ứng với phiên bản 3.12 của Python). Việc sử dụng thư mục __pycache__ giúp tránh sự lộn xộn trong việc quản lý các tệp bytecode. Quy ước đặt tên các tệp bytecode cũng giúp ngăn chặn việc các phiên bản Python khác nhau cài đặt trên cùng một máy tính ghi đè lên các tệp bytecode của nhau. Chúng ta sẽ tìm hiểu chi tiết hơn về cơ chế lưu trữ các tệp bytecode này trong Chương 22. Tuy nhiên, đây là quy trình tự động, và nó không ảnh hưởng đến hầu hết các chương trình viết bằng Python. Ngoài ra, quy ước đặt tên các tệp bytecode có thể khác nhau tùy theo phiên bản Python được sử dụng.

So why all the bother? In short, Python saves bytecode like this as a *startup-speed* optimization. The next time you run your program, Python will load the *.pyc* files and skip the compilation steps, as long as the bytecode is present, you haven't changed your source code since the bytecode was last saved, and you aren't running with a different Python than the one that created the bytecode. It works like this:

Vậy tại sao phải làm như vậy? Nói ngắn gọn, Python lưu trữ các tệp bytecode như vậy nhằm tối ưu hóa tốc độ khởi động chương trình. Lần sau khi bạn chạy chương trình, Python sẽ tải các tệp .pyc đã được lưu trữ, và bỏ qua các bước biên dịch. Điều này chỉ xảy ra nếu các tệp bytecode vẫn còn nguyên vẹn, nếu bạn không thay đổi mã nguồn kể từ lần bytecode được lưu trữ cuối cùng, và nếu bạn sử dụng chính phiên bản Python đã tạo ra các tệp bytecode đó. Cách thức hoạt động của nó như sau:

Source changes *Nguồn thay đổi*

> Python saves the last-modified timestamp and size (or hash value, optionally) of a source code file in its bytecode file, and compares this info to the source when the bytecode is loaded to know when it must recompile—if you edit and resave your source code, its bytecode is re-created the next time your program is run.
>
> Python lưu lại thông tin về thời điểm lần cuối cùng tệp mã nguồn được chỉnh sửa, kích thước của tệp đó (hoặc giá trị băm, tùy chọn), vào tệp bytecode tương ứng. Khi tệp bytecode được tải vào để thực thi, Python sẽ so sánh những thông tin này với dữ liệu trong tệp mã nguồn. Nếu bạn chỉnh sửa và lưu lại tệp mã nguồn, tệp bytecode sẽ được tạo lại vào lần chạy chương trình tiếp theo.

Python versions *Các phiên bản của Python*

> Python also adds a version-information suffix to bytecode filenames to know when it must recompile—if you run your program on a different Python implementation or version, its bytecode is generated and saved for that Python too.
>
> Python còn thêm phần mở rộng chứa thông tin về phiên bản vào tên các tệp bytecode. Điều này giúp xác định thời điểm cần biên dịch lại chương trình. Nếu bạn chạy chương trình trên một phiên bản Python khác, bytecode tương ứng cũng sẽ được tạo ra và lưu trữ dành riêng cho phiên bản Python đó.

The result is that both source code changes and differing Python versions will trigger a new bytecode file automatically. If Python *cannot* write the bytecode files to your machine, your program still works—the bytecode is generated in memory and simply discarded on program exit.

Kết quả là, cả những thay đổi trong mã nguồn lẫn việc sử dụng các phiên bản Python khác nhau đều sẽ khiến cho tệp bytecode được tạo ra một cách tự động. Nếu Python không thể ghi các tệp bytecode vào máy tính của bạn, chương trình vẫn sẽ hoạt động bình thường. Các tệp bytecode được tạo ra trong bộ nhớ và sẽ bị xóa đi ngay khi chương trình kết thúc.

Because *.pyc* files speed startup time, though, you'll want to make sure they are written for larger programs. Bytecode files are also one way to ship Python programs—Python is happy to run a program if all it can find are compatible *.pyc* files, even if the original *.py* source files are absent. This isn't as simple as deleting the *.py* files, though, and may require file moves and renames, or special techniques discussed later in Chapter 22. See Python's `compileall` module to force compiles when needed for packaging, and frozen binaries (see "Standalone Executables") for another shipping option.

Tuy nhiên, vì các tệp .pyc giúp rút ngắn thời gian khởi động chương trình, bạn nên đảm bảo rằng chúng được tạo ra dành cho các chương trình có kích thước lớn. Các tệp mã nguồn dạng bytecode cũng là một cách để phân phối các chương trình viết bằng Python. Python sẵn sàng chạy một chương trình nếu chỉ tìm thấy các tệp .pyc tương thích, ngay cả khi các tệp nguồn .py ban đầu không còn nữa. Tuy nhiên, việc này không hề đơn giản; có thể cần phải di chuyển hoặc đổi tên các tệp, hoặc sử dụng các kỹ thuật đặc biệt được trình bày trong Chương 22. Bạn có thể sử dụng mô-đun `compileall` của Python để buộc quá trình biên dịch diễn ra khi cần thiết, hoặc sử dụng các tệp nhị phân đã được biên dịch sẵn (xem mục "Các tệp thực thi độc lập") như một phương thức phân phối khác.

Strictly speaking, bytecode is an *import* optimization. Bytecode is saved in *.pyc* files only for files that are *imported*, not for the top-level files of a program that are only run as scripts. Moreover, a given file is imported and possibly compiled only *once* per program run; later imports use what's already been loaded. We'll explore import basics in Chapter 3 and take a deeper look at imports in Part V. For now, keep in mind that bytecode is never saved for code typed at the *interactive prompt*—a programming mode you'll learn about in Chapter 3 and use early in this book.

Nói một cách chính xác, bytecode thực chất là kết quả của quá trình tối ưu hóa việc nhập khẩu các đoạn mã nguồn. Bytecode chỉ được lưu trữ trong các tệp có đuôi .pyc, và chỉ đối với những tệp được nhập khẩu vào chương trình, chứ không phải đối với các tệp thuộc cấp cao nhất của chương trình – những tệp chỉ được thực thi dưới dạng script. Hơn nữa, một tệp nào đó chỉ được nhập khẩu và có thể được biên dịch một lần duy nhất trong mỗi lần chạy chương trình; các lần nhập khẩu sau đó sẽ sử dụng lại kết quả đã được lưu trữ trước đó. Chúng ta sẽ tìm hiểu cơ bản về việc nhập khẩu các đoạn mã nguồn trong Chương 3, và sẽ tìm hiểu sâu hơn về vấn đề này trong Phần V. Trong lúc này, hãy nhớ rằng bytecode không bao giờ được lưu trữ đối với các đoạn mã được gõ trực tiếp tại dòng lệnh – đây là chế độ lập trình mà bạn sẽ được tìm hiểu trong Chương 3 và sẽ sử dụng ngay từ đầu cuốn sách này.

The Python Virtual Machine (PVM)

Máy ảo Python (PVM)

Once your program has been compiled to bytecode (or the bytecode has been loaded from existing *.pyc* files), it is shipped off for execution to something generally known as the Python Virtual Machine (PVM, for acronym-inclined readers). The PVM sounds more impressive than it is; really, it's not a separate program, and it need not be installed by itself. In fact, the PVM is just a big code loop that iterates through your bytecode instructions, one by one, to carry out their operations. That is, the PVM is the runtime engine in Python. It's always present as part of the Python system, and is the component that truly runs your scripts, and is really just the last step of the "Python interpreter."

Sau khi chương trình của bạn được biên dịch thành mã bytecode (hoặc sau khi mã bytecode được tải từ các tệp .pyc đã có sẵn), chương trình đó sẽ được chuyển đến bộ máy ảo Python (Python Virtual Machine – PVM) để được thực thi. Tuy nhiên, PVM thực chất không phải là một chương trình riêng biệt; bạn cũng không cần phải cài đặt nó riêng. PVM thực chất chỉ là một vòng lặp xử lý các lệnh trong mã bytecode, thực hiện từng lệnh một. Nói cách khác, PVM chính là bộ máy thực thi trong Python. PVM luôn tồn tại như một phần của hệ thống Python; đây chính là thành phần thực sự thực hiện các chương trình được viết bằng Python. PVM cũng chính là bước cuối cùng trong quy trình thực thi chương trình bằng trình thông dịch Python.

Figure 2-1 illustrates this runtime structure. Bear in mind that all of this complexity is deliberately hidden from Python programmers. Bytecode compilation is automatic, and the PVM is just part of the Python system that you have installed on your machine. Again, programmers simply code and run files of statements, and Python handles the logistics of running them behind the scenes.

Hình 2-1 minh họa cấu trúc hoạt động của Python trong quá trình thực thi chương trình. Cần lưu ý rằng toàn bộ sự phức tạp này đều được giấu kín khỏi các lập trình viên Python. Quá trình biên dịch mã nguồn diễn ra tự động; PVM chỉ là một phần của hệ thống Python mà bạn đã cài đặt trên máy tính của mình. Lập trình viên chỉ cần viết mã và chạy các chương trình, còn Python sẽ lo liệu mọi thứ ở phía sau hậu trường.

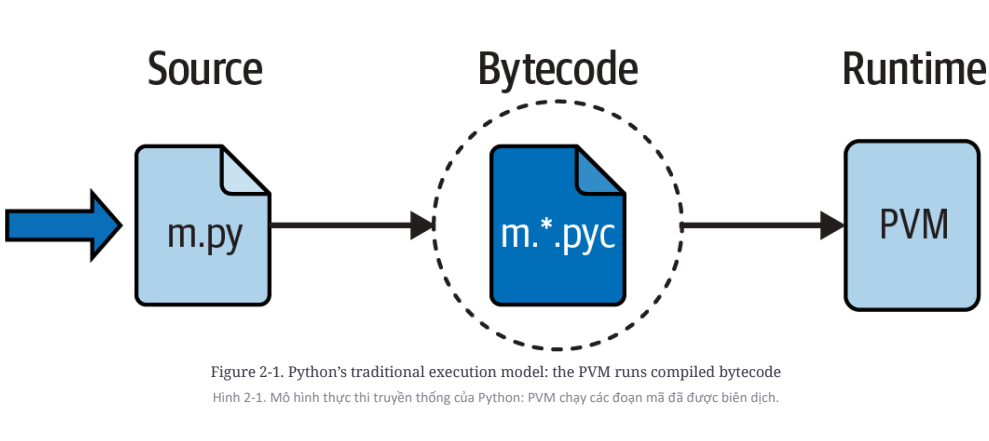

Figure 2-1. Python's traditional execution model: the PVM runs compiled bytecode
Hình 2-1. Mô hình thực thi truyền thống của Python: PVM chạy các đoạn mã đã được biên dịch.

Performance implications

Ảnh hưởng đến hiệu suất hoạt động

Readers with a background in fully compiled languages such as C and C++ might notice some glaring differences in the Python model. For one thing, there is usually no build or "make" step in Python work: code runs immediately after it is written. For another, Python bytecode is not binary *machine code* (e.g., instructions for an Intel or ARM chip, known as a *CPU*): it's a Python-specific format. There are exceptions to these rules (e.g., app builds for smartphones can take some time, and full compilers do exist, as you'll see ahead), but we're focusing on the common here first.

Những độc giả có kiến thức về các ngôn ngữ lập trình được biên dịch hoàn toàn như C và C++ có lẽ sẽ nhận thấy một số điểm khác biệt rõ rệt trong cách hoạt động của Python. Thứ nhất, trong Python, không cần qua bước biên dịch mã nguồn trước khi chạy chương trình: mã nguồn sẽ được thực thi ngay sau khi được viết ra. Thứ hai, mã bytecode của Python không phải là mã máy dạng nhị phân (tức là các lệnh dành cho chip Intel hoặc ARM, hay còn gọi là CPU). Đó là định dạng riêng của Python. Tuy nhiên, cũng có những ngoại lệ đối với các quy tắc này (ví dụ: việc biên dịch mã nguồn để sử dụng trên điện thoại thông minh có thể mất một khoảng thời gian nhất định; ngoài ra, vẫn tồn tại các trình biên dịch đầy đủ cho Python). Nhưng ở đây, chúng ta sẽ tập trung vào những đặc điểm chung của Python.

These differences explain why some Python code may not run as fast as C or C++ code, as described in Chapter 1—the PVM loop, not the CPU chip, still must interpret the bytecode, and bytecode instructions require more work than CPU instructions. On the other hand, unlike in classic interpreters, there is still an internal compile step—Python does not need to reanalyze and reparse each source statement's text repeatedly. The net effect is that pure Python code runs at speeds somewhere between those of a traditional compiled language and a traditional interpreted language.

Những khác biệt này giải thích tại sao một số đoạn mã Python có thể chạy chậm hơn so với mã viết bằng ngôn ngữ C hoặc C++. Như đã đề cập trong Chương 1, bộ xử lý vẫn phải dịch các lệnh mã nguồn thành dạng bytecode trước khi thực thi chúng. Việc xử lý các lệnh bytecode đòi hỏi nhiều công sức hơn so với việc xử lý các lệnh do bộ xử lý trực tiếp thực hiện. Mặt khác, khác với các trình thông dịch thông thường, Python vẫn có quy trình biên dịch nội bộ. Python không cần phải phân tích lại từng câu lệnh mã nguồn mỗi lần chạy chương trình. Kết quả là, mã Python chạy với tốc độ nằm giữa tốc độ của các ngôn ngữ biên dịch truyền thống và các ngôn ngữ thông dịch truyền thống.

Development implications

Ý nghĩa về mặt phát triển

Another ramification of Python's execution model is that there is really no distinction between the development and execution environments: the systems that compile and execute your source code are really one and the same. This similarity may have a bit more significance to readers with a background in traditional compiled languages, but in Python, the compiler is always present at runtime and is part of the system that runs programs.

Một hệ quả nữa của mô hình thực thi trong Python là không có sự phân biệt rõ ràng giữa môi trường phát triển và môi trường thực thi: các hệ thống dùng để biên dịch và thực thi mã nguồn thực chất là một thứ duy nhất. Điểm tương đồng này có lẽ sẽ có ý nghĩa hơn đối với những người quen với các ngôn ngữ lập trình được biên dịch truyền thống. Nhưng trong Python, trình biên dịch luôn tồn tại trong quá trình thực thi chương trình, và nó là một phần của hệ thống dùng để chạy chương trình.

This makes for a much more *rapid* development cycle. There is no need to precompile and link before execution can begin; simply type and run the code. This also adds a much more *dynamic* flavor to the language—it is possible, and often very convenient, for Python programs to construct and execute other Python programs at runtime. The `eval` and `exec` built-ins, for instance, accept and run strings containing Python program code. This structure is also why Python lends itself to product customization—because Python code can be changed on the fly, users can modify the Python parts of a system on-site without needing to compile or even possess the rest of the system's code.

Điều này giúp tăng tốc độ phát triển chương trình một cách đáng kể. Không cần phải biên dịch và kết nối các thành phần của chương trình trước khi thực thi; chỉ cần gõ và chạy đoạn mã là được. Điều này cũng khiến ngôn ngữ lập trình Python trở nên linh hoạt hơn nhiều: các chương trình Python có thể tạo ra và thực thi các chương trình Python khác ngay trong quá trình chạy. Ví dụ, các hàm tích hợp sẵn `eval` và `exec` có thể xử lý các chuỗi chứa đoạn mã Python. Cấu trúc này cũng chính là lý do tại sao Python rất thích hợp để tùy biến các sản phẩm phần mềm: vì mã Python có thể được thay đổi một cách dễ dàng, người dùng có thể chỉnh sửa các phần được viết bằng Python trong hệ thống mà không cần phải biên dịch hay nắm rõ toàn bộ mã nguồn của hệ thống đó.

At a more fundamental level, keep in mind that all we really have in Python is *runtime*—there is no initial compile-time phase at all, and everything happens as the program is running. This even includes operations such as the creation of functions and classes and the linkage of modules. Such events often occur before execution in more static languages, but happen during execution in Python. As you'll see, this makes for a much more dynamic programming experience than that to which some readers may be accustomed.

Ở mức độ cơ bản hơn, cần lưu ý rằng tất cả những gì chúng ta thực sự có trong Python chính là quá trình thực thi chương trình. Không hề có giai đoạn biên dịch ban đầu nào cả; mọi thứ đều diễn ra trong quá trình chương trình đang chạy. Điều này bao gồm cả các thao tác như tạo hàm, lớp, và kết nối các mô-đun với nhau. Trong các ngôn ngữ lập trình tĩnh, những thao tác này thường diễn ra trước khi chương trình được thực thi; còn trong Python, chúng diễn ra trong quá trình thực thi chương trình. Như bạn sẽ thấy, điều này tạo nên trải nghiệm lập trình linh hoạt hơn nhiều so với những gì mà một số người có thể quen thuộc.

WHY DOES PYTHON USE BYTECODE?
TẠI SAO PYTHON SỬ DỤNG BYTECODE?

Given that bytecode generally runs more slowly than machine code, this is a great question. The short answer is that Python uses bytecode for the sake of development speed and language flexibility.

Vì mã bytecode thường chạy chậm hơn mã máy, đây quả là một câu hỏi quan trọng. Câu trả lời ngắn gọn là: Python sử dụng mã bytecode nhằm nâng cao tốc độ phát triển và tính linh hoạt của ngôn ngữ lập trình này.

In more detail, every program must ultimately run as machine code on the host device's CPU, but program code is just text written per a language's rules. Traditional languages like C bridge this gap by constraining code to accommodate the CPU's expectations and translating the code's text to machine code ahead of time. This makes programs fast, but translation takes time, and the resulting languages are cumbersome to use.

Chi tiết hơn, mọi chương trình đều phải được thực thi dưới dạng mã máy trên bộ xử lý của thiết bị chủ. Tuy nhiên, mã chương trình thực chất chỉ là văn bản được viết theo quy tắc của ngôn ngữ lập trình. Các ngôn ngữ lập trình truyền thống như C giúp khắc phục điều này bằng cách buộc mã chương trình phải tuân theo các yêu cầu của bộ xử lý, đồng thời chuyển đổi mã văn bản thành mã máy trước khi chương trình được thực thi. Điều này giúp các chương trình hoạt động nhanh hơn. Tuy nhiên, quá trình chuyển đổi này mất nhiều thời gian, và các ngôn ngữ lập trình kết quả khá phức tạp để sử dụng.

Python instead defines an easy-to-use language that's too far removed from machine code for a direct translation and uses the PVM intermediary to run your program's bytecode on the CPU. This is a classic speed-versus-usability trade-off, but also a false dichotomy: many of the alternative implementations you'll meet ahead do compile some Python code to machine code, and Python is quick enough for many roles even with its PVM model. For more on this trade-off, see "OK, but What's the Downside?".

Thay vào đó, Python tạo ra một ngôn ngữ dễ sử dụng. Tuy nhiên, ngôn ngữ này khác quá xa so với mã máy, nên không thể được dịch trực tiếp sang mã máy. Để chạy chương trình viết bằng Python trên CPU, người ta sử dụng công cụ PVM. Đây là một trường hợp phải đánh đổi giữa tốc độ và tính dễ sử dụng. Nhưng thực ra đây không phải là một sự đối lập hoàn toàn: nhiều phương pháp triển khai khác cũng có thể biên dịch mã Python thành mã máy. Hơn nữa, Python vẫn đủ nhanh để đáp ứng nhu cầu sử dụng trong nhiều trường hợp, ngay cả khi sử dụng mô hình PVM. Để biết thêm về sự đánh đổi này, hãy xem mục "Được rồi, nhưng điểm yếu của Python là gì?".

Execution-Model Variations

Các biến thể của mô hình thực thi

Now that you've studied the internal execution flow described in the prior section, you should also know that it reflects just the standard implementation of Python today and is not a requirement of the Python language itself. Because of that, the execution model is prone to change with time. In fact, many systems already modify the picture in Figure 2-1 in one way or another. Before moving on, let's briefly explore the most prominent of these variations.

Sau khi đã tìm hiểu về quy trình thực thi bên trong được mô tả ở phần trước, bạn cũng nên biết rằng đó chỉ là cách triển khai tiêu chuẩn của ngôn ngữ Python hiện nay mà thôi; đó không phải là yêu cầu bắt buộc của chính ngôn ngữ Python. Do đó, mô hình thực thi này có thể thay đổi theo thời gian. Thực tế, nhiều hệ thống đã điều chỉnh cấu trúc được minh họa trong Hình 2-1 theo nhiều cách khác nhau. Trước khi tiếp tục, hãy cùng tìm hiểu ngắn gọn về những thay đổi nổi bật nhất trong số này.

Python Implementation Alternatives

Các phương án triển khai bằng Python

As of this writing, there are multiple implementations of the Python language. Although there is much cross-fertilization of ideas and work between these Pythons, each is a separately installed software system, with its own project and user base. All but one of these systems are optional reading for most Python beginners, but a quick look at the ways they modify the execution model might help demystify what it means to run code in general.

Tính đến thời điểm hiện tại, có nhiều phiên bản khác nhau của ngôn ngữ lập trình Python. Mặc dù có sự chia sẻ ý tưởng và kết quả công việc giữa các phiên bản Python này, nhưng mỗi phiên bản đều là một hệ thống phần mềm riêng biệt, với dự án và cộng đồng người dùng riêng. Hầu hết các phiên bản này đều không bắt buộc phải được cài đặt đối với những người mới bắt đầu học Python. Tuy nhiên, nếu tìm hiểu sơ lược về cách các phiên bản này thay đổi cách thức thực thi mã nguồn, chúng ta sẽ dễ hiểu hơn về quy trình thực thi mã lập trình nói chung.

The short story is that *CPython* is the standard implementation—what we've called the "common" version so far. It's the usual Python on PCs and smartphones, and the system that most readers will be using (and if you're not sure, this probably includes you). This is also the version used in this book, though the Python language fundamentals presented here apply to all the alternatives too. All the other Python implementations have specific goals and may or may not implement the full Python language defined by CPython, but come close enough to qualify as Pythons.

Nói một cách ngắn gọn, CPython chính là phiên bản tiêu chuẩn của ngôn ngữ Python. Đây chính là phiên bản Python mà hầu hết mọi người đều sử dụng trên máy tính cá nhân và điện thoại thông minh. Nếu bạn chưa chắc chắn, có lẽ bạn cũng thuộc nhóm người sử dụng phiên bản này. Phiên bản này cũng được sử dụng trong cuốn sách này. Tuy nhiên, những kiến thức cơ bản về ngôn ngữ Python được trình bày ở đây vẫn có thể áp dụng cho tất cả các phiên bản khác. Các phiên bản Python khác đều có mục đích sử dụng cụ thể; chúng có thể hoặc không thể thực hiện đầy đủ các tính năng của ngôn ngữ Python như được định nghĩa bởi CPython. Nhưng dù sao thì chúng vẫn có thể được coi là các phiên bản của ngôn ngữ Python.

For example, *PyPy* is a drop-in replacement for CPython that runs many programs quicker by compiling parts of them further as they run. *Jython* and *IronPython* instead reimplement CPython to provide access to Java and .NET components; although standard CPython programs can access such components too (e.g., via *pyjnius* and *Chaquopy* for Java), Jython and IronPython aim to be more seamless. Other options accelerate numeric code, or code in general.

Ví dụ, PyPy là công cụ có thể thay thế hoàn toàn cho CPython. Nó giúp các chương trình chạy nhanh hơn nhờ việc biên dịch các đoạn mã trong quá trình chương trình đang được thực thi. Trong khi đó, Jython và IronPython lại tái triển khai CPython nhằm tạo điều kiện để truy cập vào các thành phần của nền tảng Java và .NET. Mặc dù các chương trình viết bằng CPython thông thường cũng có thể truy cập vào các thành phần này (ví dụ: thông qua pyjnius và Chaquopy dành cho Java), nhưng mục tiêu của Jython và IronPython là mang lại trải nghiệm sử dụng mượt mà hơn. Ngoài ra, còn có nhiều công cụ khác giúp tăng tốc độ thực thi các đoạn mã số học, hay nói chung là các đoạn mã nói chung.

In more detail, the following list is a quick rundown on the most prominent Python implementations available today—with the usual apologies to current options omitted for space, and future options omitted for lack of a crystal ball:

Chi tiết hơn, danh sách dưới đây là bản tóm tắt nhanh về các phiên bản Python phổ biến nhất hiện nay. Các lựa chọn hiện có được bỏ qua để tiết kiệm không gian; còn các lựa chọn trong tương lai thì không được đề cập do khó có thể dự đoán trước được.

CPython: the standard *CPython: Tiêu chuẩn hàng đầu*

> The original, standard, and reference implementation of Python is usually called CPython when you want to contrast it with the other options (and just plain "Python" otherwise). This name comes from the fact that it is coded in portable ANSI C language code. This is the Python that you fetch from *python.org* for PCs, get with most alternative distributions and Linux repos, and have inside Python apps on Android and iOS smartphones. This is also the flavor whose implementation is captured in Figure 2-1, though this is prone to change (and in fact may: see "Future Possibilities").
>
> Phiên bản gốc, tiêu chuẩn của Python thường được gọi là CPython, nhằm phân biệt nó với các phiên bản khác của Python. Tên gọi này bắt nguồn từ việc CPython được viết bằng ngôn ngữ lập trình ANSI C – một ngôn ngữ có khả năng chạy trên nhiều nền tảng khác nhau. Đây chính là phiên bản Python mà chúng ta có thể tải về từ trang web python.org dành cho máy tính cá nhân; nó cũng có sẵn trong hầu hết các bản phân phối Linux. Ngoài ra, phiên bản này còn được sử dụng trong các ứng dụng Python trên điện thoại thông minh chạy hệ điều hành Android và iOS. Phiên bản CPython cũng chính là phiên bản được minh họa trong Hình 2-1. Tuy nhiên, phiên bản này có thể thay đổi theo thời gian (xem mục "Khả năng thay đổi trong tương lai").

Jython: Python for Java

Jython: Phiên bản Python dành cho môi trường Java

> The Jython system is an alternative implementation of the Python language. It's targeted at integration with the Java programming language, much as CPython integrates with C and C++ components. Jython consists of Java classes that compile Python source code to Java bytecode, which is then routed to the Java Virtual Machine (JVM). Programmers still code Python statements in *.py* text files as usual; the Jython system essentially just replaces the rightmost two bubbles in Figure 2-1 with Java-based equivalents for seamless Java linkage. At this writing, Jython implements the older Python 2.X not used in this book but is working toward 3.X.
>
> Hệ thống Jython là một phiên bản thay thế của ngôn ngữ lập trình Python. Nó được thiết kế để kết hợp với ngôn ngữ lập trình Java, tương tự như cách mà CPython kết hợp với các thành phần lập trình bằng ngôn ngữ C và C++. Jython gồm các lớp Java, những lớp này chuyển đổi mã nguồn Python thành mã bytecode của Java. Mã bytecode này sau đó được chuyển đến Máy ảo Java (JVM) để được thực thi. Các lập trình viên vẫn viết các đoạn mã Python trong các tệp văn bản có đuôi .py như thông thường. Về cơ bản, Jython chỉ thay thế hai phần được chỉ ra trong Hình 2-1 bằng các thành phần tương ứng của ngôn ngữ Java, nhằm đảm bảo sự kết nối liền mạch với hệ thống Java. Tại thời điểm viết bài này, Jython hỗ trợ phiên bản Python 2.X – phiên bản không được sử dụng trong cuốn sách này. Tuy nhiên, Jython đang trong quá trình hỗ trợ phiên bản 3.X.

IronPython: Python for .NET

IronPython: Phiên bản Python dành cho môi trường .NET

> IronPython is another alternative implementation of Python. Coded in C#, it is designed to allow Python programs to integrate with applications written to work with Microsoft's .NET Framework for Windows, as well as the Mono open source equivalent. Like Jython, IronPython replaces the last two bubbles in Figure 2-1 with equivalents for execution in the .NET environment. With it, Python code can gain accessibility both to and from other .NET languages and leverage .NET libraries.
>
> IronPython là một phiên bản thay thế khác của ngôn ngữ lập trình Python. Được viết bằng ngôn ngữ C#, IronPython được thiết kế nhằm giúp các chương trình viết bằng Python có thể tích hợp với các ứng dụng được viết để hoạt động trên nền tảng .NET Framework của Microsoft, cũng như các ứng dụng được viết bằng công nghệ Mono. Giống như Jython, IronPython thay thế hai phần được chỉ ra trong Hình 2-1 bằng các thành phần tương ứng để thực thi chương trình trong môi trường .NET. Nhờ vậy, mã nguồn Python có thể tương tác dễ dàng với các ngôn ngữ lập trình khác trong nền tảng .NET, đồng thời tận dụng được các thư viện của .NET.

Stackless: Python for concurrency

Stackless: Python dùng để xử lý các tác vụ đồng thời

> The Stackless Python system is an enhanced version of the standard CPython language oriented toward concurrency. Because it does not save state on the C language call stack, Stackless can make Python easier to port to small-stack architectures. Stackless also provides efficient multiprocessing options that some find more straightforward than CPython's later `async` coroutines, and fosters novel programming structures. As an example, the game *EVE Online* uses Stackless Python to achieve high performance for massively parallel tasks.
>
> Hệ thống Stackless Python là phiên bản được nâng cấp từ ngôn ngữ lập trình CPython tiêu chuẩn, được thiết kế để phù hợp với các tình huống yêu cầu xử lý đa luồng. Vì không lưu trữ trạng thái của chương trình trên stack gọi hàm của ngôn ngữ C, nên Stackless giúp việc chuyển đổi chương trình viết bằng Python sang các hệ thống có cấu trúc bộ nhớ nhỏ trở nên dễ dàng hơn. Stackless còn cung cấp các tính năng xử lý đa luồng hiệu quả; một số người cho rằng những tính năng này đơn giản hơn so với các cơ chế xử lý đa luồng được tích hợp trong CPython. Ngoài ra, Stackless còn hỗ trợ việc tạo ra các cấu trúc lập trình mới mẻ. Ví dụ, trò chơi EVE Online sử dụng Stackless Python để đạt được hiệu suất cao trong việc xử lý các tác vụ đa luồng.

PyPy: a JIT compiler for speed

PyPy: Một trình biên dịch kiểu JIT nhằm nâng cao tốc độ thực thi chương trình.

> PyPy is a reimplementation of Python for speed. It was one of the first systems to employ a just-in-time (*JIT*) compiler for normal Python code. A JIT compiler is just an extension to the PVM—the rightmost bubble in Figure 2-1—that translates portions of your bytecode all the way to machine code for faster execution. PyPy does this as your program is running, not in a prerun compile step, and is able to create type-specific machine code for the dynamic Python language by keeping track of the data types of the objects your program processes. By replacing portions of your bytecode this way, your program runs faster and faster as it is executing. In addition, some Python programs may also take up less memory under PyPy.
>
> PyPy là phiên bản được tái triển khai của ngôn ngữ Python nhằm nâng cao tốc độ thực thi chương trình. Đây là một trong những hệ thống đầu tiên sử dụng trình biên dịch kiểu "just-in-time" (JIT) để xử lý các đoạn mã Python thông thường. Trình biên dịch JIT thực chất chỉ là một phần mở rộng của PVM – thành phần nằm bên phải nhất trong Hình 2-1. Trình biên dịch này chuyển đổi các đoạn mã bytecode thành mã máy, giúp chương trình được thực thi nhanh hơn. PyPy thực hiện việc này trong quá trình chương trình đang chạy, chứ không phải trước khi chương trình được thực thi. Ngoài ra, PyPy còn có khả năng tạo ra mã máy phù hợp với từng kiểu dữ liệu cụ thể, nhờ việc theo dõi các kiểu dữ liệu của các đối tượng mà chương trình xử lý. Bằng cách thay thế một số đoạn mã bytecode theo cách này, chương trình sẽ chạy nhanh hơn. Thêm vào đó, một số chương trình viết bằng Python có thể sử dụng ít bộ nhớ hơn khi chạy trên PyPy.

Numba: a JIT compiler for numeric speed

Numba: Một trình biên dịch kiểu JIT nhằm nâng cao tốc độ xử lý các phép toán số học.

> The Numba extension for Python adds a JIT compiler that optimizes numerically oriented code by compiling it all the way to machine code while your program runs. To direct Numba's compiler, functions are augmented with Python "@" decorators supplied with the Numba install. While not all Python code can be sped up by Numba, it works well for code that uses *NumPy* arrays and functions, as well as math-oriented loops. Numba also supports code parallelization paradigms commonly used in scientific programming.
>
> Tiện ích mở rộng Numba dành cho Python bổ sung thêm bộ biên dịch JIT, giúp tối ưu hóa các đoạn mã liên quan đến các phép tính số học. Quá trình biên dịch diễn ra ngay trong lúc chương trình đang chạy, và kết quả là mã máy được tạo ra. Để điều khiển quá trình biên dịch này, các hàm trong chương trình được gắn thêm các tiền tố "@" do chính Numba cung cấp. Mặc dù không phải mọi đoạn mã Python đều có thể được tối ưu hóa bằng Numba, nhưng công cụ này rất hiệu quả đối với các đoạn mã sử dụng các mảng và hàm của thư viện NumPy, cũng như các vòng lặp liên quan đến các phép tính toán học. Numba còn hỗ trợ các phương thức xử lý song song, vốn thường được sử dụng trong lập trình khoa học.

Shed Skin: an AOT compiler for conforming code

Shed Skin: Một trình biên dịch AOT dùng để tạo ra mã nguồn tuân thủ các quy định.

> Shed Skin is an ahead-of-time (*AOT*) compiler that translates unadorned Python code to C++ code, which is then compiled to machine code before it is run. With an AOT, the two rightmost bubbles in Figure 2-1 are replaced with precompiled machine code. Shed Skin can yield both standalone programs and extension modules for use in other programs. In exchange, it implements a restricted subset of Python that requires Python variables to meet an implicit statically typed constraint and does not support some Python features or libraries today. Nevertheless, Shed Skin may outperform both CPython and JIT-based options for some conforming code.
>
> Shed Skin là một trình biên dịch kiểu AOT (Ahead-Of-Time). Nó chuyển đổi mã nguồn Python thành mã nguồn C++. Sau đó, mã C++ này được biên dịch thành mã máy trước khi được thực thi. Nhờ cơ chế AOT này, hai phần tử nằm bên phải nhất trong Hình 2-1 được thay thế bằng mã máy đã được biên dịch sẵn. Shed Skin có thể tạo ra cả các chương trình độc lập lẫn các mô-đun có thể được sử dụng trong các chương trình khác. Tuy nhiên, Shed Skin chỉ hỗ trợ một tập hợp các tính năng hạn chế của Python; các biến trong chương trình Python phải tuân theo các quy tắc kiểu dữ liệu cứng nhắc. Ngoài ra, Shed Skin không hỗ trợ một số tính năng hoặc thư viện của Python hiện nay. Dù vậy, đối với một số đoạn mã tuân thủ quy ước của Python, Shed Skin có thể hoạt động hiệu quả hơn cả CPython và các giải pháp dựa trên công nghệ JIT.

PyThran: an AOT compiler for numeric speed

PyThran: Một trình biên dịch kiểu AOT nhằm nâng cao tốc độ xử lý các phép toán số học.

> PyThran implements another AOT compiler for a subset of the Python language, with a focus on scientific programming. Like Shed Skin, it translates Python code to C++, using static type declarations provided in either formatted comments or separate command files. The result of compiling the generated C++ is a native module that can be imported and used by other Python code.
>
> PyThran là công cụ biên dịch kiểu AOT, được thiết kế để xử lý một tập con của ngôn ngữ Python, với trọng tâm vào lĩnh vực lập trình khoa học. Giống như Shed Skin, PyThran cũng chuyển đổi mã nguồn Python sang ngôn ngữ C++. Việc khai báo kiểu dữ liệu được thực hiện thông qua các chú thích có cấu trúc hoặc các tệp lệnh riêng biệt. Kết quả của quá trình biên dịch là các mô-đun có thể được các đoạn mã Python khác sử dụng.

Cython: a Python/C hybrid for speed

Cython: Một nền tảng lai giữa Python và C, nhằm nâng cao tốc độ thực thi chương trình.

> The Cython system defines a Python/C hybrid language that combines Python code with the ability to call C functions and use C type declarations for variables, parameters, and class attributes. Cython code can be AOT-compiled to C code that uses the Python/C API, which may then be compiled completely to machine code. Though not compatible with standard Python, Cython can be useful both for wrapping external C libraries and for implementing performance-critical parts of a system as efficient C extensions for use in CPython programs.
>
> Hệ thống Cython tạo ra một ngôn ngữ lai giữa Python và C. Ngôn ngữ này kết hợp khả năng sử dụng mã Python với khả năng gọi các hàm trong ngôn ngữ C, đồng thời cho phép sử dụng các kiểu dữ liệu của ngôn ngữ C cho các biến, tham số và thuộc tính của lớp. Mã Cython có thể được biên dịch thành mã C sử dụng giao diện API của Python/C; sau đó, mã C này có thể được biên dịch thành mã máy. Mặc dù không tương thích với Python tiêu chuẩn, nhưng Cython vẫn rất hữu ích trong việc tích hợp các thư viện C bên ngoài vào chương trình, hoặc trong việc triển khai các phần quan trọng về mặt hiệu năng của hệ thống dưới dạng các chương trình mở rộng bằng ngôn ngữ C, để có thể được sử dụng trong các chương trình viết bằng Python.

MicroPython: a Python subset for constraints

MicroPython: Phiên bản thu gọn của Python, được thiết kế để đáp ứng các yêu cầu cụ thể.

> The MicroPython system is an alternative Python with a focus on efficiency. It implements a limited dialect of the CPython language and a small subset of its standard library, in order to optimize Python to run in constrained environments. Though originally targeted at microcontrollers, MicroPython is also compiled for the *WebAssembly* system you'll meet in Chapter 3, to support Python programs in web browsers without the full heft of CPython.
>
> Hệ thống MicroPython là một phiên bản thay thế của ngôn ngữ Python, tập trung vào khả năng hoạt động hiệu quả trong các môi trường có điều kiện hạn chế. Nó sử dụng một phiên bản giới hạn của ngôn ngữ CPython, cùng một tập hợp các hàm thuộc thư viện tiêu chuẩn của CPython, nhằm giúp Python hoạt động tốt hơn trong những điều kiện đó. Mặc dù ban đầu được thiết kế dành cho các bộ điều khiển vi mạch, MicroPython cũng có thể được biên dịch để hoạt động trên nền tảng WebAssembly – điều mà chúng ta sẽ tìm hiểu chi tiết trong Chương 3. Nhờ vậy, các chương trình viết bằng MicroPython có thể chạy trên trình duyệt mà không cần phải sử dụng toàn bộ khả năng của CPython.

And (naturally) more *Và (dĩ nhiên) còn nhiều thứ nữa.*

> For other Python alternatives, see the list at Python's wiki, as well as the results of a web search. Among the others, *Cinder* is a performance-focused implementation of CPython with a JIT compiler and more, created by Meta and used for Instagram; *Pyston* is a fork of CPython with a JIT compiler and other optimizations, started by Dropbox; and *Nuitka* is a free and paid optimizing AOT compiler that translates standard Python code to C, and is used by the emerging *py2wasm* to translate Python code to *WebAssembly* for use in browsers.
>
> Đối với các giải pháp thay thế Python khác, bạn có thể tham khảo danh sách trên trang wiki của Python, cũng như kết quả tìm kiếm trên mạng. Trong số đó, Cinder là phiên bản CPython được tối ưu hóa về mặt hiệu năng, sử dụng trình biên dịch JIT và nhiều tính năng khác nữa. Cinder được phát triển bởi Meta và được sử dụng trong ứng dụng Instagram. Pyston là phiên bản được phát triển từ CPython, cũng sử dụng trình biên dịch JIT và các công cụ tối ưu hóa khác. Pyston được phát triển bởi Dropbox. Nuitka là trình biên dịch miễn phí hoặc có phí, có khả năng chuyển đổi mã Python thành mã nguồn C. Nuitka được sử dụng trong công cụ py2wasm, công cụ chuyển đổi mã Python thành WebAssembly để sử dụng trong các trình duyệt.

All that being said, unless you have a specific need met only by one of the alternatives, you'll probably want to use the standard CPython system. Because it is the reference implementation of the language, it's always the most complete, and also tends to be more up-to-date and robust than others. Still, it's unlikely that CPython will ever subsume all the optimization projects afoot in the Python world, so be sure to vet the alternatives when your goals gel.

Tuy nhiên, trừ khi bạn có nhu cầu cụ thể mà chỉ có một trong các giải pháp thay thế này mới có thể đáp ứng được, nếu không thì có lẽ bạn nên sử dụng hệ thống CPython tiêu chuẩn. Vì đây là phiên bản mẫu của ngôn ngữ lập trình này, nên nó luôn đầy đủ chức năng nhất. Ngoài ra, CPython còn thường được cập nhật thường xuyên và có độ ổn định cao hơn các giải pháp thay thế khác. Tuy nhiên, khó có khả năng CPython có thể thay thế tất cả các giải pháp tối ưu hóa hiện có trong thế giới Python. Vì vậy, hãy cân nhắc kỹ các giải pháp thay thế khác trước khi đưa ra quyết định cuối cùng.

Standalone Executables Các tệp thực thi độc lập

Sometimes when people ask for a "real" Python compiler, what they're actually seeking is simply a way to generate standalone executables from their Python programs. This is more a packaging and shipping topic than an execution-flow concept—and of more interest to software developers than Python beginners—but it's a related idea.

Đôi khi, khi mọi người yêu cầu một trình biên dịch Python "thực sự", thì thực chất họ chỉ muốn có cách để chuyển các chương trình Python thành các tệp thực thi có thể chạy được một cách độc lập. Đây thực chất là vấn đề liên quan đến việc đóng gói và phân phối chương trình, chứ không phải là khái niệm liên quan đến quy trình thực thi chương trình. Đây là vấn đề mà các nhà phát triển phần mềm quan tâm nhiều hơn là những người mới bắt đầu học Python. Tuy nhiên, đây vẫn là một ý tưởng có liên quan đến việc sử dụng Python.

In short, with the help of third-party tools that you can fetch off the web, it is possible to turn your Python programs into true self-contained executables, sometimes also called *frozen binaries* in the Python world. Whatever they're called, these programs can be run without requiring a Python installation or shipping your source code files.

Nói ngắn gọn, với sự trợ giúp của các công cụ từ bên thứ ba có thể tải về từ mạng, chúng ta có thể biến các chương trình Python thành những tệp thực thi hoàn chỉnh. Trong thế giới Python, những tệp thực thi này đôi khi còn được gọi là "binary files". Dù được gọi bằng cái tên nào đi nữa, những chương trình này đều có thể được chạy mà không cần phải cài đặt Python hay gửi kèm các tệp mã nguồn của chúng.

Standalone executables bundle together the bytecode of your program files, along with the PVM (interpreter) and any Python support files and libraries your program needs, into a single package. There are some variations on this theme, but the end result can be a single *executable* (e.g., a *.exe* file on Windows) or *app* (e.g., a *.app* on macOS, and *.apk* or *.aab* on Android) that can easily be shipped to customers. In Figure 2-1, it is as though the two rightmost bubbles—bytecode and PVM—are merged into a single component: a standalone executable bundle.

Các tệp thực thi độc lập gộp chung mã byte của các tệp chương trình, cùng với công cụ PVM (trình thông dịch) và mọi tệp hỗ trợ cũng như thư viện mà chương trình cần thiết, thành một gói duy nhất. Có nhiều biến thể của cấu trúc này, nhưng kết quả cuối cùng vẫn là một tệp thực thi đơn lẻ (ví dụ: tệp .exe trên Windows) hoặc một ứng dụng (ví dụ: tệp .app trên macOS, hoặc tệp .apk hoặc .aab trên Android). Những tệp này có thể được gửi đến khách hàng một cách dễ dàng. Trong Hình 2-1, có vẻ như hai thành phần ở bên phải nhất – mã byte và PVM – được hợp nhất thành một thành phần duy nhất, tức là tệp thực thi độc lập.

Today, a variety of systems are capable of generating standalone executables and vary in platforms and features. For example, *py2exe* builds standalones for Windows; *PyInstaller* and *cx_freeze* make them for Windows, macOS, and Linux; *py2app* creates them for macOS; and *Buildozer* and *Briefcase* generate apps for Android and iOS.

Ngày nay, có nhiều công cụ khác nhau có khả năng tạo ra các chương trình có thể chạy độc lập trên nhiều nền tảng khác nhau. Ví dụ, py2exe dùng để tạo chương trình có thể chạy trên Windows; PyInstaller và cx_freeze dùng để tạo chương trình có thể chạy trên Windows, macOS và Linux; py2app dùng để tạo chương trình có thể chạy trên macOS; còn Buildozer và Briefcase thì dùng để tạo ứng dụng cho Android và iOS.

Frozen binaries are not the same as the output of a true compiler—they run bytecode through a virtual machine. Hence, apart from a possible startup improvement, frozen binaries run at the same speed as the original source files. Frozen binaries are also not generally small (they contain a PVM), but by current standards they are not unusually large either. Because Python is embedded in the frozen bundle, though, it does not have to be installed on the receiving end to run your program. Moreover, because your bytecode is embedded in the bundle, it isn't as easily viewed.

Các tệp nhị phân dạng "frozen" không giống với kết quả xuất ra bởi trình biên dịch thông thường. Các tệp này chạy bằng cách sử dụng mã byte thông qua máy ảo. Do đó, ngoại trừ việc có thể giúp quá trình khởi động diễn ra nhanh hơn, các tệp nhị phân dạng "frozen" vẫn chạy với tốc độ tương tự như các tệp nguồn gốc. Thông thường, các tệp nhị phân dạng "frozen" khá lớn (vì chúng chứa cả thành phần PVM), nhưng theo các tiêu chuẩn hiện nay, kích thước của chúng cũng không quá lớn. Tuy nhiên, vì Python được tích hợp sẵn trong tệp nhị phân dạng "frozen", nên người dùng không cần phải cài đặt Python trên máy tính để chạy chương trình của mình. Hơn nữa, vì mã byte được tích hợp sẵn trong tệp nhị phân, nên mã này khó có thể bị xem trực tiếp hơn.

For more details, see the alternative coverage of standalones in this book's Appendix A, which includes tips on building them on multiple platforms from the same code base.

Để biết thêm chi tiết, xin xem phần mô tả về cách xây dựng các ứng dụng độc lập trong Phụ lục A của cuốn sách này. Phần này cũng cung cấp các mẹo hữu ích về cách xây dựng các ứng dụng trên nhiều nền tảng khác nhau, từ cùng một bộ mã nguồn.

Future Possibilities Những khả năng trong tương lai

Finally, note that the runtime execution model sketched here is really an artifact of the current implementation of Python, not of the language itself. For instance, it's possible that an AOT compiler for translating unrestricted and unadorned Python source code to machine code may appear during the shelf life of this book (although the fact that one has not in over three decades makes this seem unlikely!), and JIT compilers seem to be cropping up everywhere.

Cuối cùng, cần lưu ý rằng mô hình thực thi chương trình được mô tả ở đây thực chất chỉ là kết quả của cách triển khai Python hiện tại, chứ không phải là đặc điểm cố hữu của chính ngôn ngữ Python. Ví dụ, có thể trong thời gian cuốn sách này còn được sử dụng, sẽ xuất hiện các trình biên dịch kiểu AOT, những công cụ có khả năng chuyển đổi mã nguồn Python thành mã máy một cách trực tiếp. Tuy nhiên, việc này dường như khó xảy ra, vì đã hơn ba thập kỷ trôi qua mà vẫn chưa có trình biên dịch nào như vậy được phát triển. Trong khi đó, các trình biên dịch kiểu JIT ngày càng trở nên phổ biến.

Either way, the bytecode model will likely be standard for some time to come. The portability and flexibility of bytecode are important features of many Python systems. Moreover, adding type-constraint declarations to support static compilation would break much of the flexibility, conciseness, simplicity, and spirit of Python coding we're about to explore. Python's type hinting, also covered later, comes close, but is thankfully still unused by Python itself today.

Dù thế nào đi nữa, có lẽ mô hình mã byte sẽ tiếp tục là tiêu chuẩn trong một thời gian dài nữa. Khả năng di chuyển và tính linh hoạt của mã byte là những đặc điểm quan trọng của nhiều hệ thống lập trình bằng Python. Hơn nữa, việc thêm các quy định về kiểu dữ liệu nhằm hỗ trợ quá trình biên dịch tĩnh sẽ làm mất đi phần lớn tính linh hoạt, sự ngắn gọn và đơn giản vốn có của ngôn ngữ lập trình Python. Cơ chế gợi ý kiểu dữ liệu trong Python cũng tương tự, nhưng may mắn thay, tính năng này hiện vẫn chưa được Python sử dụng.

NOTE GHI CHÚ

JIT futurism: While *CPython* currently follows the bytecode/PVM model in Figure 2-1, it may augment it in the future. Version 3.13, still under development as this is being written, will add an experimental JIT compiler. As in *PyPy* and others, this will translate some bytecode all the way to machine code as your program is running. In 3.13 it will have a negligible speed boost and will be disabled by default. Though prescience is perilous in publishing, the JIT compiler may be enabled by default for CPython in the future if it ever yields a significant net gain for Python programs.

Chủ nghĩa tương lai của công nghệ JIT: Hiện tại, CPython vẫn sử dụng mô hình bytecode/PVM như được minh họa trong Hình 2-1. Tuy nhiên, trong tương lai, CPython có thể sẽ cải tiến mô hình này. Phiên bản 3.13, vẫn đang trong quá trình phát triển, sẽ bổ sung tính năng biên dịch JIT. Giống như PyPy và các công cụ tương tự, tính năng này sẽ chuyển đổi một phần bytecode thành mã máy ngay trong lúc chương trình đang chạy. Trong phiên bản 3.13, tốc độ xử lý sẽ được cải thiện đáng kể, nhưng tính năng này sẽ được tắt theo mặc định. Mặc dù khó có thể dự đoán chính xác tương lai, nhưng có lẽ trong tương lai, tính năng biên dịch JIT sẽ được bật theo mặc định cho CPython, nếu nó thực sự mang lại lợi ích đáng kể cho các chương trình viết bằng Python.

Chapter Summary Tóm tắt chương

This chapter introduced the execution model of Python—how Python runs your programs—and explored some common variations on that model designed for both different roles and better performance. Although you don't really need to come to grips with Python internals to write Python scripts, a passing acquaintance with this chapter's topics will help you truly understand how your programs run once you start coding them, as you will in the next chapter. First though, here's the usual chapter quiz to review what you've learned so far.

Chương này giới thiệu về mô hình thực thi của Python – cách mà Python chạy các chương trình của bạn. Đồng thời, chương này cũng trình bày một số biến thể phổ biến của mô hình này, nhằm phục vụ các mục đích sử dụng khác nhau và nâng cao hiệu suất hoạt động của chương trình. Mặc dù bạn không nhất thiết phải hiểu rõ cấu trúc nội bộ của Python để có thể viết các chương trình bằng ngôn ngữ này, nhưng việc nắm vững những nội dung trong chương này sẽ giúp bạn hiểu rõ hơn cách các chương trình của mình được thực thi, khi bạn bắt đầu lập trình. Trước khi đi vào nội dung tiếp theo, hãy cùng làm bài kiểm tra nhỏ để ôn tập lại những gì bạn đã học được cho đến nay.

Test Your Knowledge: Quiz

Kiểm tra kiến thức của bạn: Bài trắc nghiệm

1. What is the Python interpreter?
   Python interpreter là gì?
2. What is source code? Nguyên mẫu mã nguồn là gì?
3. What is bytecode? Bytecode là gì?
4. What is the PVM? PVM là gì?
5. What is machine code? Mã máy là gì?
6. Name two or more variations on Python's standard execution model.
   Hãy nêu ra hai hoặc nhiều biến thể của mô hình thực thi tiêu chuẩn trong Python.
7. How are CPython, Jython, and IronPython different?
   CPython, Jython và IronPython khác nhau như thế nào?
8. What are PyPy, Shed Skin, and Cython?
   PyPy, Shed Skin và Cython là gì?

Test Your Knowledge: Answers

Kiểm tra kiến thức của bạn: Đáp án

1. The Python interpreter is a program that runs the Python programs you write. It essentially intermediates between your Python instructions and those available in a CPU's machine code.
   Trình thông dịch Python là chương trình dùng để thực thi các chương trình viết bằng ngôn ngữ Python. Về bản chất, trình thông dịch này đóng vai trò trung gian giữa các lệnh Python mà bạn viết và các lệnh máy mà bộ xử lý CPU sử dụng để thực hiện các thao tác.
2. Source code is the statements you write for your program. It consists of text in text files whose names normally end with a *.py* extension.
   Mã nguồn chính là những lệnh mà bạn viết để tạo nên chương trình của mình. Mã nguồn được lưu trữ dưới dạng văn bản trong các tệp tin, và tên các tệp tin này thường kết thúc bằng phần mở rộng .py.
3. Bytecode is the lower-level form of your program after Python compiles it. Python automatically stores bytecode in files with a *.pyc* extension when possible, and automatically re-creates it when needed.
   Bytecode là dạng mã ở cấp độ thấp của chương trình sau khi được Python biên dịch. Khi có thể, Python sẽ lưu bytecode vào các tệp có phần mở rộng .pyc. Ngoài ra, Python cũng tự động tạo lại bytecode khi cần thiết.
4. The PVM is the Python Virtual Machine—the runtime engine of Python that interprets your compiled bytecode.
   PVM chính là Python Virtual Machine – nền tảng thực thi chương trình của ngôn ngữ lập trình Python, có chức năng dịch các đoạn mã đã được biên dịch thành dạng bytecode.
5. Machine code is the low-level instructions of the underlying CPU on a computing device like a PC or phone. Because this is what every program ultimately runs, Python code must be translated to this by a software layer like the PVM interpreter, or JIT or AOT compilers.
   Mã máy là những lệnh ở cấp độ thấp mà bộ xử lý trung tâm của các thiết bị tính toán như máy tính cá nhân hay điện thoại di động phải thực hiện. Vì đây chính là những lệnh mà mọi chương trình phải thực hiện, nên mã Python cần được chuyển đổi thành mã máy bởi các công cụ phần mềm như trình thông dịch PVM, hoặc các trình biên dịch kiểu JIT/AOT.
6. Numba, Shed Skin, and standalone executables are all variations on the execution model. In addition, the alternative implementations of Python named in the next two answers modify the model in some fashion as well—by replacing bytecode and VMs, or by adding JIT and AOT compilers.
   Numba, Shed Skin và các chương trình thực thi độc lập đều là những biến thể của mô hình thực thi này. Ngoài ra, các phiên bản Python khác cũng làm thay đổi mô hình thực thi này theo nhiều cách khác nhau: bằng cách thay thế mã bytecode và máy ảo, hoặc bằng cách bổ sung các công cụ biên dịch kiểu JIT và AOT.
7. CPython is the standard and reference implementation of the language. Jython and IronPython process Python programs for use in Java and .NET environments, respectively; they are alternative compilers for Python.
   CPython là phiên bản tiêu chuẩn và được sử dụng làm chuẩn mực để triển khai ngôn ngữ lập trình Python. Jython và IronPython là những công cụ dùng để chạy các chương trình Python trong môi trường Java và .NET tương ứng; chúng có thể được coi là những trình biên dịch thay thế cho Python.
8. PyPy and Shed Skin are reimplementations of Python targeted at speed. PyPy speeds normal Python programs by using runtime type information and a JIT compiler to replace some Python bytecode with machine code as the program runs. Shed Skin speeds programs with an AOT compiler that translates a restricted subset of Python to C++, from which it can be fully compiled to machine code to be run as a program or used in others. Cython is a Python/C combination that can be compiled into machine-code extensions accessible to CPython code.
   PyPy và Shed Skin là những phiên bản được tái triển khai của ngôn ngữ Python, nhằm mục đích nâng cao tốc độ thực thi chương trình. PyPy tăng tốc độ thực thi các chương trình Python bằng cách sử dụng thông tin về kiểu dữ liệu trong quá trình thực thi chương trình, cùng với trình biên dịch JIT để chuyển đổi một phần mã bytecode của Python thành mã máy. Shed Skin thì sử dụng trình biên dịch AOT để chuyển đổi một tập hợp các lệnh trong Python sang ngôn ngữ C++. Sau đó, mã C++ này được biên dịch thành mã máy, có thể được sử dụng trực tiếp hoặc kết hợp với các chương trình khác. Cython là công cụ kết hợp giữa Python và C; mã viết bằng Cython có thể được biên dịch thành các đoạn mã máy mà CPython có thể sử dụng được.